

LEARNING FROM EXAMPLES

In which we describe agents that can improve their behavior through diligent study of past experiences and predictions about the future.

An agent is **learning** if it improves its performance after making observations about the world. Learning can range from the trivial, such as jotting down a shopping list, to the profound, as when Albert Einstein inferred a new theory of the universe. When the agent is a computer, we call it **machine learning**: a computer observes some data, builds a **model** based on the data, and uses the model as both a **hypothesis** about the world and a piece of software that can solve problems.

Machine learning

Why would we want a machine to learn? Why not just program it the right way to begin with? There are two main reasons. First, the designers cannot anticipate all possible future situations. For example, a robot designed to navigate mazes must learn the layout of each new maze it encounters; a program for predicting stock market prices must learn to adapt when conditions change from boom to bust. Second, sometimes the designers have no idea how to program a solution themselves. Most people are good at recognizing the faces of family members, but they do it subconsciously, so even the best programmers don't know how to program a computer to accomplish that task, except by using machine learning algorithms.

In this chapter, we interleave a discussion of various model classes—decision trees (Section 19.3), linear models (Section 19.6), nonparametric models such as nearest neighbors (Section 19.7), ensemble models such as random forests (Section 19.8)—with practical advice on building machine learning systems (Section 19.9), and discussion of the theory of machine learning (Sections 19.1 to 19.5).

19.1 Forms of Learning

Any component of an agent program can be improved by machine learning. The improvements, and the techniques used to make them, depend on these factors:

- Which *component* is to be improved.
- What *prior knowledge* the agent has, which influences the *model* it builds.
- What *data* and *feedback* on that data is available.

Chapter 2 described several agent designs. The **components** of these agents include:

1. A direct mapping from conditions on the current state to actions.
2. A means to infer relevant properties of the world from the percept sequence.
3. Information about the way the world evolves and about the results of possible actions the agent can take.

4. Utility information indicating the desirability of world states.
5. Action-value information indicating the desirability of actions.
6. Goals that describe the most desirable states.
7. A problem generator, critic, and learning element that enable the system to improve.

Each of these components can be learned. Consider a self-driving car agent that learns by observing a human driver. Every time the driver brakes, the agent might learn a condition–action rule for when to brake (component 1). By seeing many camera images that it is told contain buses, it can learn to recognize them (component 2). By trying actions and observing the results—for example, braking hard on a wet road—it can learn the effects of its actions (component 3). Then, when it receives complaints from passengers who have been thoroughly shaken up during the trip, it can learn a useful component of its overall utility function (component 4).

The technology of machine learning has become a standard part of software engineering. Any time you are building a software system, even if you don’t think of it as an AI agent, components of the system can potentially be improved with machine learning. For example, software to analyze images of galaxies under gravitational lensing was speeded up by a factor of 10 million with a machine-learned model (Hezaveh *et al.*, 2017), and energy use for cooling data centers was reduced by 40% with another machine-learned model (Gao, 2014). Turing Award winner David Patterson and Google AI head Jeff Dean declared the dawn of a “Golden Age” for computer architecture due to machine learning (Dean *et al.*, 2018).

We have seen several examples of models for agent components: atomic, factored, and relational models based on logic or probability, and so on. Learning algorithms have been devised for all of these.

Prior knowledge

This chapter assumes little **prior knowledge** on the part of the agent: it starts from scratch and learns from the data. In Section 21.7.2 we consider **transfer learning**, in which knowledge from one domain is transferred to a new domain, so that learning can proceed faster with less data. We do assume, however, that the designer of the system chooses a model framework that can lead to effective learning.

Going from a specific set of observations to a general rule is called **induction**; from the observations that the sun rose every day in the past, we induce that the sun will come up tomorrow. This differs from the **deduction** we studied in Chapter 7 because the inductive conclusions may be incorrect, whereas deductive conclusions are guaranteed to be correct if the premises are correct.

This chapter concentrates on problems where the input is a **factored representation**—a vector of attribute values. It is also possible for the input to be any kind of data structure, including atomic and relational.

When the output is one of a finite set of values (such as *sunny/cloudy/rainy* or *true/false*), the learning problem is called **classification**. When it is a number (such as tomorrow’s temperature, measured either as an integer or a real number), the learning problem has the (admittedly obscure¹) name **regression**.

Classification

Regression

¹ A better name would have been *function approximation* or *numeric prediction*. But in 1886 Francis Galton wrote an influential article on the concept of *regression to the mean* (e.g., the children of tall parents are likely to be taller than average, but not as tall as the parents). Galton showed plots with what he called “regression lines,” and readers came to associate the word “regression” with the statistical technique of function approximation rather than with the topic of regression to the mean.

There are three types of **feedback** that can accompany the inputs, and that determine the three main types of learning: Feedback

- In **supervised learning** the agent observes input-output pairs and learns a function that maps from input to output. For example, the inputs could be camera images, each one accompanied by an output saying “bus” or “pedestrian,” etc. An output like this is called a **label**. The agent learns a function that, when given a new image, predicts the appropriate label. In the case of braking actions (component 1 above), an input is the current state (speed and direction of the car, road condition), and an output is the distance it took to stop. In this case a set of output values can be obtained by the agent from its own percepts (after the fact); the environment is the teacher, and the agent learns a function that maps states to stopping distance. Supervised learning
Label
- In **unsupervised learning** the agent learns patterns in the input without any explicit feedback. The most common unsupervised learning task is **clustering**: detecting potentially useful clusters of input examples. For example, when shown millions of images taken from the Internet, a computer vision system can identify a large cluster of similar images which an English speaker would call “cats.” Unsupervised learning
- In **reinforcement learning** the agent learns from a series of reinforcements: rewards and punishments. For example, at the end of a chess game the agent is told that it has won (a reward) or lost (a punishment). It is up to the agent to decide which of the actions prior to the reinforcement were most responsible for it, and to alter its actions to aim towards more rewards in the future. Reinforcement learning

19.2 Supervised Learning

More formally, the task of supervised learning is this:

Given a **training set** of N example input–output pairs Training set

$$(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N),$$

where each pair was generated by an unknown function $y = f(x)$,
discover a function h that approximates the true function f .

The function h is called a **hypothesis** about the world. It is drawn from a **hypothesis space** $\mathcal{H}$ of possible functions. For example, the hypothesis space might be the set of polynomials of degree 3; or the set of Javascript functions; or the set of 3-SAT Boolean logic formulas. Hypothesis space

With alternative vocabulary, we can say that h is a **model** of the data, drawn from a **model class** $\mathcal{H}$, or we can say a **function** drawn from a **function class**. We call the output y_i the **ground truth**—the true answer we are asking our model to predict. Model class
Ground truth

How do we choose a hypothesis space? We might have some prior knowledge about the process that generated the data. If not, we can perform **exploratory data analysis**: examining the data with statistical tests and visualizations—histograms, scatter plots, box plots—to get a feel for the data, and some insight into what hypothesis space might be appropriate. Or we can just try multiple hypothesis spaces and evaluate which one works best. Exploratory data analysis

How do we choose a good hypothesis from within the hypothesis space? We could hope for a **consistent hypothesis**: and h such that each x_i in the training set has $h(x_i) = y_i$. With continuous-valued outputs we can’t expect an exact match to the ground truth; instead we Consistent hypothesis

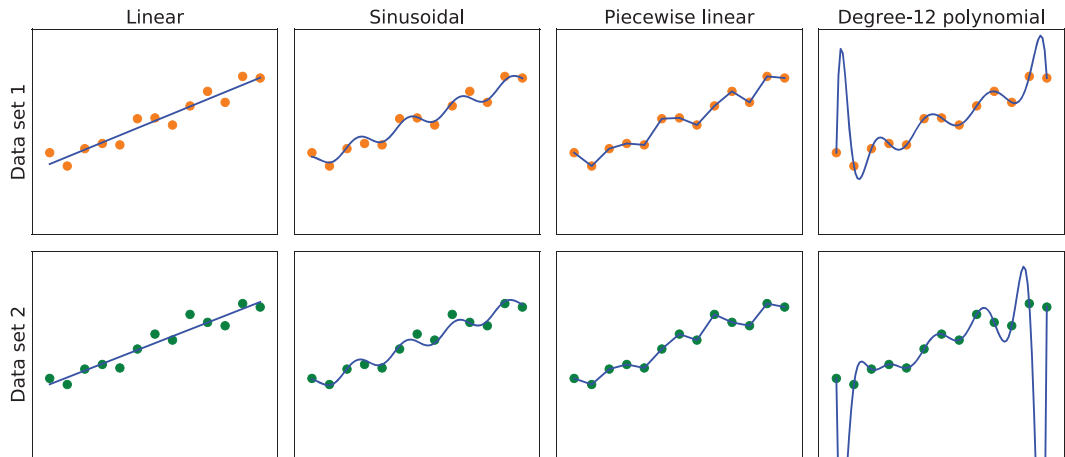


Figure 19.1 Finding hypotheses to fit data. **Top row:** four plots of best-fit functions from four different hypothesis spaces trained on data set 1. **Bottom row:** the same four functions, but trained on a slightly different data set (sampled from the same $f(x)$ function).

look for a **best-fit function** for which each $h(x_i)$ is close to y_i (in a way that we will formalize in Section 19.4.2).

The true measure of a hypothesis is not how it does on the training set, but rather how well it handles inputs it has not yet seen. We can evaluate that with a second sample of (x_i, y_i) pairs called a **test set**. We say that h **generalizes** well if it accurately predicts the outputs of the test set.

Figure 19.1 shows that the function h that a learning algorithm discovers depends on the hypothesis space $\mathcal{H}$ it considers and on the training set it is given. Each of the four plots in the top row have the same training set of 13 data points in the (x, y) plane. The four plots in the bottom row have a second set of 13 data points; both sets are representative of the same unknown function $f(x)$. Each column shows the best-fit hypothesis h from a different hypothesis space:

- **Column 1:** Straight lines; functions of the form $h(x) = w_1x + w_0$. There is no line that would be a consistent hypothesis for the data points.
- **Column 2:** Sinusoidal functions of the form $h(x) = w_1x + \sin(w_0x)$. This choice is not quite consistent, but fits both data sets very well.
- **Column 3:** Piecewise-linear functions where each line segment connects the dots from one data point to the next. These functions are always consistent.
- **Column 4:** Degree-12 polynomials, $h(x) = \sum_{i=0}^{12} w_i x^i$. These are consistent: we can always get a degree-12 polynomial to perfectly fit 13 distinct points. But just because the hypothesis is consistent does not mean it is a good guess.

One way to analyze hypothesis spaces is by the bias they impose (regardless of the training data set) and the variance they produce (from one training set to another).

By **bias** we mean (loosely) the tendency of a predictive hypothesis to deviate from the expected value when averaged over different training sets. Bias often results from restrictions

Test set
Generalization

Bias

imposed by the hypothesis space. For example, the hypothesis space of linear functions induces a strong bias: it only allows functions consisting of straight lines. If there are any patterns in the data other than the overall slope of a line, a linear function will not be able to represent those patterns. We say that a hypothesis is **underfitting** when it fails to find a pattern in the data. On the other hand, the piecewise linear function has low bias; the shape of the function is driven by the data.

Underfitting

By **variance** we mean the amount of change in the hypothesis due to fluctuation in the training data. The two rows of Figure 19.1 represent data sets that were each sampled from the same $f(x)$ function. The data sets turned out to be slightly different. For the first three columns, the small difference in the data set translates into a small difference in the hypothesis. We call that low variance. But the degree-12 polynomials in the fourth column have high variance: look how different the two functions are at both ends of the x -axis. Clearly, at least one of these polynomials must be a poor approximation to the true $f(x)$. We say a function is **overfitting** the data when it pays too much attention to the particular data set it is trained on, causing it to perform poorly on unseen data.

Variance

Overfitting

Often there is a **bias–variance tradeoff**: a choice between more complex, low-bias hypotheses that fit the training data well and simpler, low-variance hypotheses that may generalize better. Albert Einstein said in 1933, “the supreme goal of all theory is to make the irreducible basic elements as simple and as few as possible without having to surrender the adequate representation of a single datum of experience.” In other words, Einstein recommends choosing the simplest hypothesis that matches the data. This principle can be traced further back to the 14th-century English philosopher William of Ockham.² His principle that “plurality [of entities] should not be posited without necessity” is called **Ockham’s razor** because it is used to “shave off” dubious explanations.

Bias–variance tradeoff

Defining simplicity is not easy. It seems clear that a polynomial with only two parameters is simpler than one with thirteen parameters. We will make this intuition more precise in Section 19.3.4. However, in Chapter 21 we will see that deep neural network models can often generalize quite well, even though they are very complex—some of them have billions of parameters. So the number of parameters by itself is not a good measure of a model’s fitness. Perhaps we should be aiming for “appropriateness,” not “simplicity” in a model class. We will consider this issue in Section 19.4.1.

Which hypothesis is best in Figure 19.1? We can’t be certain. If we knew the data represented, say, the number of hits to a Web site that grows from day to day, but also cycles depending on the time of day, then we might favor the sinusoidal function. If we knew the data was definitely not cyclic but had high noise, that would favor the linear function.

In some cases, an analyst is willing to say not just that a hypothesis is possible or impossible, but rather how probable it is. Supervised learning can be done by choosing the hypothesis h^* that is most probable given the data:

$$h^* = \operatorname{argmax}_{h \in \mathcal{H}} P(h|\text{data}).$$

By Bayes’ rule this is equivalent to

$$h^* = \operatorname{argmax}_{h \in \mathcal{H}} P(\text{data}|h) P(h).$$

² The name is often misspelled as “Occam.”

Then we can say that the prior probability $P(h)$ is high for a smooth degree-1 or -2 polynomial and lower for a degree-12 polynomial with large, sharp spikes. We allow unusual-looking functions when the data say we really need them, but we discourage them by giving them a low prior probability.

► Why not let $\mathcal{H}$ be the class of all computer programs, or all Turing machines? The problem is that *there is a tradeoff between the expressiveness of a hypothesis space and the computational complexity of finding a good hypothesis within that space*. For example, fitting a straight line to data is an easy computation; fitting high-degree polynomials is somewhat harder; and fitting Turing machines is undecidable. A second reason to prefer simple hypothesis spaces is that presumably we will want to use h after we have learned it, and computing $h(x)$ when h is a linear function is guaranteed to be fast, while computing an arbitrary Turing machine program is not even guaranteed to terminate.

For these reasons, most work on learning has focused on simple representations. In recent years there has been great interest in deep learning (Chapter 21), where representations are not simple but where the $h(x)$ computation still takes only a *bounded number of steps* to compute with appropriate hardware.

We will see that the expressiveness–complexity tradeoff is not simple: it is often the case, as we saw with first-order logic in Chapter 8, that an expressive language makes it possible for a *simple* hypothesis to fit the data, whereas restricting the expressiveness of the language means that any consistent hypothesis must be complex.

19.2.1 Example problem: Restaurant waiting

We will describe a sample supervised learning problem in detail: the problem of deciding whether to wait for a table at a restaurant. This problem will be used throughout the chapter to demonstrate different model classes. For this problem the output, y , is a Boolean variable that we will call *WillWait*; it is true for examples where we do wait for a table. The input, x , is a vector of ten attribute values, each of which has discrete values:

1. *Alternate*: whether there is a suitable alternative restaurant nearby.
2. *Bar*: whether the restaurant has a comfortable bar area to wait in.
3. *Fri/Sat*: true on Fridays and Saturdays.
4. *Hungry*: whether we are hungry right now.
5. *Patrons*: how many people are in the restaurant (values are *None*, *Some*, and *Full*).
6. *Price*: the restaurant's price range (\$, \$\$, \$\$\$).
7. *Raining*: whether it is raining outside.
8. *Reservation*: whether we made a reservation.
9. *Type*: the kind of restaurant (French, Italian, Thai, or burger).
10. *WaitEstimate*: host's wait estimate: 0–10, 10–30, 30–60, or >60 minutes.

A set of 12 examples, taken from the experience of one of us (SR), is shown in Figure 19.2. Note how skimpy these data are: there are $2^6 \times 3^2 \times 4^2 = 9,216$ possible combinations of values for the input attributes, but we are given the correct output for only 12 of them; each of the other 9,204 could be either true or false; we don't know. This is the essence of induction: we need to make our best guess at these missing 9,204 output values, given only the evidence of the 12 examples.

Example	Input Attributes										Output
	<i>Alt</i>	<i>Bar</i>	<i>Fri</i>	<i>Hun</i>	<i>Pat</i>	<i>Price</i>	<i>Rain</i>	<i>Res</i>	<i>Type</i>	<i>Est</i>	<i>WillWait</i>
x_1	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>Yes</i>	<i>Some</i>	<i>\$\$\$</i>	<i>No</i>	<i>Yes</i>	<i>French</i>	<i>0–10</i>	$y_1 = \text{Yes}$
x_2	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>Yes</i>	<i>Full</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Thai</i>	<i>30–60</i>	$y_2 = \text{No}$
x_3	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>Some</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Burger</i>	<i>0–10</i>	$y_3 = \text{Yes}$
x_4	<i>Yes</i>	<i>No</i>	<i>Yes</i>	<i>Yes</i>	<i>Full</i>	<i>\$</i>	<i>Yes</i>	<i>No</i>	<i>Thai</i>	<i>10–30</i>	$y_4 = \text{Yes}$
x_5	<i>Yes</i>	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>Full</i>	<i>\$\$\$</i>	<i>No</i>	<i>Yes</i>	<i>French</i>	<i>>60</i>	$y_5 = \text{No}$
x_6	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>Yes</i>	<i>Some</i>	<i>\$\$</i>	<i>Yes</i>	<i>Yes</i>	<i>Italian</i>	<i>0–10</i>	$y_6 = \text{Yes}$
x_7	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>None</i>	<i>\$</i>	<i>Yes</i>	<i>No</i>	<i>Burger</i>	<i>0–10</i>	$y_7 = \text{No}$
x_8	<i>No</i>	<i>No</i>	<i>No</i>	<i>Yes</i>	<i>Some</i>	<i>\$\$</i>	<i>Yes</i>	<i>Yes</i>	<i>Thai</i>	<i>0–10</i>	$y_8 = \text{Yes}$
x_9	<i>No</i>	<i>Yes</i>	<i>Yes</i>	<i>No</i>	<i>Full</i>	<i>\$</i>	<i>Yes</i>	<i>No</i>	<i>Burger</i>	<i>>60</i>	$y_9 = \text{No}$
x_{10}	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Full</i>	<i>\$\$\$</i>	<i>No</i>	<i>Yes</i>	<i>Italian</i>	<i>10–30</i>	$y_{10} = \text{No}$
x_{11}	<i>No</i>	<i>No</i>	<i>No</i>	<i>No</i>	<i>None</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Thai</i>	<i>0–10</i>	$y_{11} = \text{No}$
x_{12}	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Full</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Burger</i>	<i>30–60</i>	$y_{12} = \text{Yes}$

Figure 19.2 Examples for the restaurant domain.

19.3 Learning Decision Trees

A **decision tree** is a representation of a function that maps a vector of attribute values to a single output value—a “decision.” A decision tree reaches its decision by performing a sequence of tests, starting at the root and following the appropriate branch until a leaf is reached. Each internal node in the tree corresponds to a test of the value of one of the input attributes, the branches from the node are labeled with the possible values of the attribute, and the leaf nodes specify what value is to be returned by the function.

Decision tree

In general, the input and output values can be discrete or continuous, but for now we will consider only inputs consisting of discrete values and outputs that are either *true* (a **positive** example) or *false* (a **negative** example). We call this **Boolean classification**. We will use j to index the examples (x_j is the input vector for the j th example and y_j is the output), and $x_{j,i}$ for the i th attribute of the j th example.

Positive
Negative

The tree representing the decision function that SR uses for the restaurant problem is shown in Figure 19.3. Following the branches, we see that an example with *Patrons* = *Full* and *WaitEstimate* = 0–10 will be classified as positive (i.e., yes, we will wait for a table).

19.3.1 Expressiveness of decision trees

A Boolean decision tree is equivalent to a logical statement of the form:

$$\text{Output} \Leftrightarrow (\text{Path}_1 \vee \text{Path}_2 \vee \dots),$$

where each Path_i is a conjunction of the form $(A_m = v_x \wedge A_n = v_y \wedge \dots)$ of attribute-value tests corresponding to a path from the root to a *true* leaf. Thus, the whole expression is in disjunctive normal form, which means that any function in propositional logic can be expressed as a decision tree.

For many problems, the decision tree format yields a nice, concise, understandable result. Indeed, many “How To” manuals (e.g., for car repair) are written as decision trees. But some functions cannot be represented concisely. For example, the majority function, which returns

true if and only if more than half of the inputs are true, requires an exponentially large decision tree, as does the parity function, which returns true if and only if an even number of input attributes are true. With real-valued attributes, the function $y > A_1 + A_2$ is hard to represent with a decision tree because the decision boundary is a diagonal line, and all decision tree tests divide the space up into rectangular, axis-aligned boxes. We would have to stack a lot of boxes to closely approximate the diagonal line. In other words, decision trees are good for some kinds of functions and bad for others.

Is there *any* kind of representation that is efficient for *all* kinds of functions? Unfortunately, the answer is no—there are just too many functions to be able to represent them all with a small number of bits. Even just considering Boolean functions with n Boolean attributes, the truth table will have 2^n rows, and each row can output *true* or *false*, so there are 2^{2^n} different functions. With 20 attributes there are $2^{1,048,576} \approx 10^{300,000}$ functions, so if we limit ourselves to a million-bit representation, we can't represent all these functions.

19.3.2 Learning decision trees from examples

We want to find a tree that is consistent with the examples in Figure 19.2 and is as small as possible. Unfortunately, it is intractable to find a guaranteed smallest consistent tree. But with some simple heuristics, we can efficiently find one that is close to the smallest. The LEARN-DECISION-TREE algorithm adopts a greedy divide-and-conquer strategy: always test the most important attribute first, then recursively solve the smaller subproblems that are defined by the possible results of the test. By “most important attribute,” we mean the one that makes the most difference to the classification of an example. That way, we hope to get to the correct classification with a small number of tests, meaning that all paths in the tree will be short and the tree as a whole will be shallow.

Figure 19.4(a) shows that *Type* is a poor attribute, because it leaves us with four possible

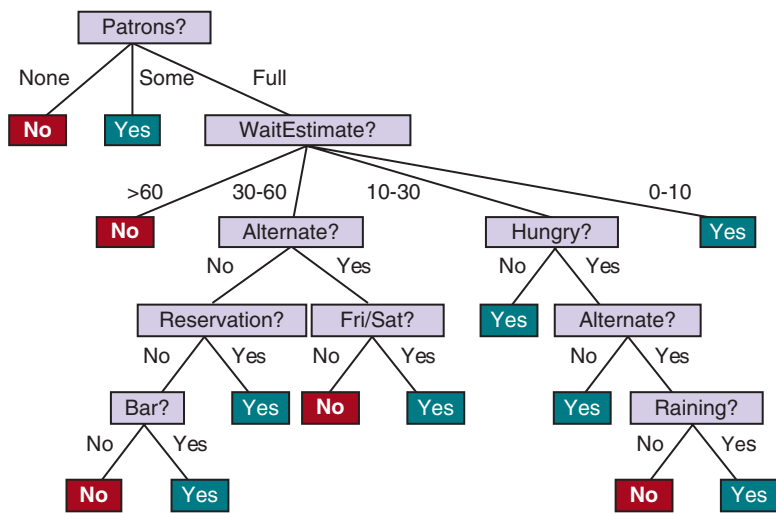


Figure 19.3 A decision tree for deciding whether to wait for a table.

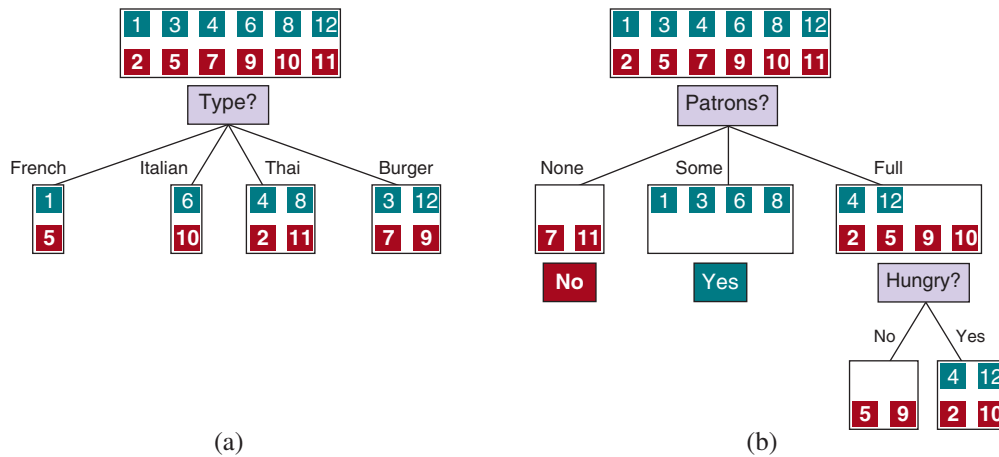


Figure 19.4 Splitting the examples by testing on attributes. At each node we show the positive (light boxes) and negative (dark boxes) examples remaining. (a) Splitting on *Type* brings us no nearer to distinguishing between positive and negative examples. (b) Splitting on *Patrons* does a good job of separating positive and negative examples. After splitting on *Patrons*, *Hungry* is a fairly good second test.

outcomes, each of which has the same number of positive as negative examples. On the other hand, in (b) we see that *Patrons* is a fairly important attribute, because if the value is *None* or *Some*, then we are left with example sets for which we can answer definitively (*No* and *Yes*, respectively). If the value is *Full*, we are left with a mixed set of examples. There are four cases to consider for these recursive subproblems:

1. If the remaining examples are all positive (or all negative), then we are done: we can answer *Yes* or *No*. Figure 19.4(b) shows examples of this happening in the *None* and *Some* branches.
2. If there are some positive and some negative examples, then choose the best attribute to split them. Figure 19.4(b) shows *Hungry* being used to split the remaining examples.
3. If there are no examples left, it means that no example has been observed for this combination of attribute values, and we return the most common output value from the set of examples that were used in constructing the node's parent.
4. If there are no attributes left, but both positive and negative examples, it means that these examples have exactly the same description, but different classifications. This can happen because there is an error or **noise** in the data; because the domain is nondeterministic; or because we can't observe an attribute that would distinguish the examples. The best we can do is return the most common output value of the remaining examples.

Noise

The LEARN-DECISION-TREE algorithm is shown in Figure 19.5. Note that the set of examples is an input to the algorithm, but nowhere do the examples appear in the tree returned by the algorithm. A tree consists of tests on attributes in the interior nodes, values of attributes on the branches, and output values on the leaf nodes. The details of the IMPORTANCE function are given in Section 19.3.3. The output of the learning algorithm on our sample training set is shown in Figure 19.6. The tree is clearly different from the original tree shown in Fig-

```

function LEARN-DECISION-TREE(examples, attributes, parent_examples) returns a tree
  if examples is empty then return PLURALITY-VALUE(parent_examples)
  else if all examples have the same classification then return the classification
  else if attributes is empty then return PLURALITY-VALUE(examples)
  else
     $A \leftarrow \operatorname{argmax}_{a \in \text{attributes}} \text{IMPORTANCE}(a, \text{examples})$ 
    tree  $\leftarrow$  a new decision tree with root test A
    for each value v of A do
      exs  $\leftarrow \{e : e \in \text{examples} \text{ and } e.A = v\}$ 
      subtree  $\leftarrow$  LEARN-DECISION-TREE(exs, attributes − A, examples)
      add a branch to tree with label (A = v) and subtree subtree
  return tree

```

Figure 19.5 The decision tree learning algorithm. The function IMPORTANCE is described in Section 19.3.3. The function PLURALITY-VALUE selects the most common output value among a set of examples, breaking ties randomly.

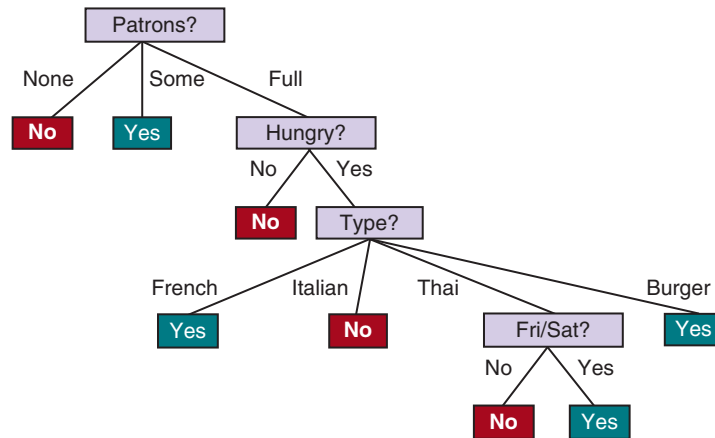


Figure 19.6 The decision tree induced from the 12-example training set.

ure 19.3. One might conclude that the learning algorithm is not doing a very good job of learning the correct function. This would be the wrong conclusion to draw, however. The learning algorithm looks at the *examples*, not at the correct function, and in fact, its hypothesis (see Figure 19.6) not only is consistent with all the examples, but is considerably simpler than the original tree! With slightly different examples the tree might be very different, but the function it represents would be similar.

The learning algorithm has no reason to include tests for *Raining* and *Reservation*, because it can classify all the examples without them. It has also detected an interesting and previously unsuspected pattern: SR will wait for Thai food on weekends. It is also bound to make some mistakes for cases where it has seen no examples. For example, it has never seen

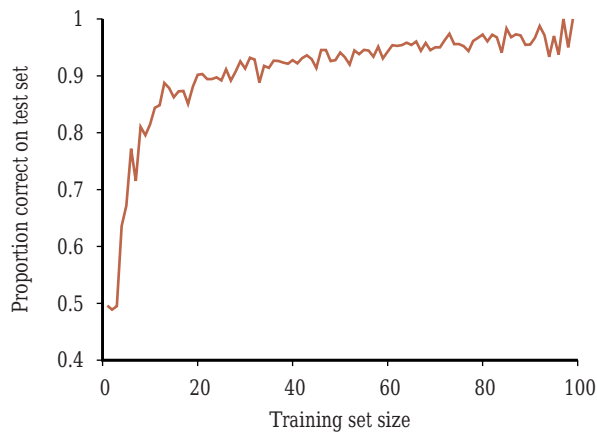


Figure 19.7 A learning curve for the decision tree learning algorithm on 100 randomly generated examples in the restaurant domain. Each data point is the average of 20 trials.

a case where the wait is 0–10 minutes but the restaurant is full. In that case it says not to wait when *Hungry* is false, but SR would certainly wait. With more training examples the learning program could correct this mistake.

We can evaluate the performance of a learning algorithm with a **learning curve**, as shown in Figure 19.7. For this figure we have 100 examples at our disposal, which we split randomly into a training set and a test set. We learn a hypothesis h with the training set and measure its accuracy with the test set. We can do this starting with a training set of size 1 and increasing one at a time up to size 99. For each size, we actually repeat the process of randomly splitting into training and test sets 20 times, and average the results of the 20 trials. The curve shows that as the training set size grows, the accuracy increases. (For this reason, learning curves are also called **happy graphs**.) In this graph we reach 95% accuracy, and it looks as if the curve might continue to increase if we had more data.

Learning curve

Happy graphs

19.3.3 Choosing attribute tests

The decision tree learning algorithm chooses the attribute with the highest IMPORTANCE. We will now show how to measure importance, using the notion of information gain, which is defined in terms of **entropy**, which is the fundamental quantity in information theory (Shannon and Weaver, 1949).

Entropy

Entropy is a measure of the uncertainty of a random variable; the more information, the less entropy. A random variable with only one possible value—a coin that always comes up heads—has no uncertainty and thus its entropy is defined as zero. A fair coin is equally likely to come up heads or tails when flipped, and we will soon show that this counts as “1 bit” of entropy. The roll of a fair *four*-sided die has 2 bits of entropy, because there are 2^2 equally probable choices. Now consider an unfair coin that comes up heads 99% of the time. Intuitively, this coin has less uncertainty than the fair coin—if we guess heads we’ll be wrong only 1% of the time—so we would like it to have an entropy measure that is close to zero, but positive. In general, the entropy of a random variable V with values v_k having probability

$P(v_k)$ is defined as

$$\text{Entropy: } H(V) = \sum_k P(v_k) \log_2 \frac{1}{P(v_k)} = - \sum_k P(v_k) \log_2 P(v_k).$$

We can check that the entropy of a fair coin flip is indeed 1 bit:

$$H(\text{Fair}) = -(0.5 \log_2 0.5 + 0.5 \log_2 0.5) = 1.$$

And of a four-sided die is 2 bits:

$$H(\text{Die4}) = -(0.25 \log_2 0.25 + 0.25 \log_2 0.25 + 0.25 \log_2 0.25 + 0.25 \log_2 0.25) = 2$$

For the loaded coin with 99% heads, we get

$$H(\text{Loaded}) = -(0.99 \log_2 0.99 + 0.01 \log_2 0.01) \approx 0.08 \text{ bits.}$$

It will help to define $B(q)$ as the entropy of a Boolean random variable that is true with probability q :

$$B(q) = -(q \log_2 q + (1 - q) \log_2 (1 - q)).$$

Thus, $H(\text{Loaded}) = B(0.99) \approx 0.08$. Now let's get back to decision tree learning. If a training set contains p positive examples and n negative examples, then the entropy of the output variable on the whole set is

$$H(\text{Output}) = B\left(\frac{p}{p+n}\right).$$

The restaurant training set in Figure 19.2 has $p = n = 6$, so the corresponding entropy is $B(0.5)$ or exactly 1 bit. The result of a test on an attribute A will give us some information, thus reducing the overall entropy by some amount. We can measure this reduction by looking at the entropy remaining after the attribute test.

An attribute A with d distinct values divides the training set E into subsets $E_1, \dots, E_d$. Each subset E_k has p_k positive examples and n_k negative examples, so if we go along that branch, we will need an additional $B(p_k/(p_k + n_k))$ bits of information to answer the question. A randomly chosen example from the training set has the k th value for the attribute (i.e., is in E_k with probability $(p_k + n_k)/(p + n)$), so the expected entropy remaining after testing attribute A is

$$\text{Remainder}(A) = \sum_{k=1}^d \frac{p_k + n_k}{p + n} B\left(\frac{p_k}{p_k + n_k}\right).$$

Information gain

The **information gain** from the attribute test on A is the expected reduction in entropy:

$$\text{Gain}(A) = B\left(\frac{p}{p+n}\right) - \text{Remainder}(A).$$

In fact $\text{Gain}(A)$ is just what we need to implement the IMPORTANCE function. Returning to the attributes considered in Figure 19.4, we have

$$\text{Gain}(\text{Patrons}) = 1 - \left[\frac{2}{12} B\left(\frac{0}{2}\right) + \frac{4}{12} B\left(\frac{4}{4}\right) + \frac{6}{12} B\left(\frac{2}{6}\right) \right] \approx 0.541 \text{ bits,}$$

$$\text{Gain}(\text{Type}) = 1 - \left[\frac{2}{12} B\left(\frac{1}{2}\right) + \frac{2}{12} B\left(\frac{1}{2}\right) + \frac{4}{12} B\left(\frac{2}{4}\right) + \frac{4}{12} B\left(\frac{2}{4}\right) \right] = 0 \text{ bits,}$$

confirming our intuition that *Patrons* is a better attribute to split on first. In fact, *Patrons* has the maximum information gain of any of the attributes and thus would be chosen by the decision tree learning algorithm as the root.

19.3.4 Generalization and overfitting

We want our learning algorithms to find a hypothesis that fits the training data, but more importantly, we want it to generalize well for previously unseen data. In Figure 19.1 we saw that a high-degree polynomial can fit all the data, but has wild swings that are not warranted by the data: it fits but can overfit. Overfitting becomes more likely as the number of attributes grows, and less likely as we increase the number of training examples. Larger hypothesis spaces (e.g., decision trees with more nodes or polynomials with high degree) have more capacity both to fit and to overfit; some model classes are more prone to overfitting than others.

For decision trees, a technique called **decision tree pruning** combats overfitting. Pruning works by eliminating nodes that are not clearly relevant. We start with a full tree, as generated by LEARN-DECISION-TREE. We then look at a test node that has only leaf nodes as descendants. If the test appears to be irrelevant—detecting only noise in the data—then we eliminate the test, replacing it with a leaf node. We repeat this process, considering each test with only leaf descendants, until each one has either been pruned or accepted as is.

Decision tree pruning

The question is, how do we detect that a node is testing an irrelevant attribute? Suppose we are at a node consisting of p positive and n negative examples. If the attribute is irrelevant, we would expect that it would split the examples into subsets such that each subset has roughly the same proportion of positive examples as the whole set, $p/(p+n)$, and so the information gain will be close to zero.³ Thus, a low information gain is a good clue that the attribute is irrelevant. Now the question is, how large a gain should we require in order to split on a particular attribute?

We can answer this question by using a statistical **significance test**. Such a test begins by assuming that there is no underlying pattern (the so-called **null hypothesis**). Then the actual data are analyzed to calculate the extent to which they deviate from a perfect absence of pattern. If the degree of deviation is statistically unlikely (usually taken to mean a 5% probability or less), then that is considered to be good evidence for the presence of a significant pattern in the data. The probabilities are calculated from standard distributions of the amount of deviation one would expect to see in random sampling.

Significance test
Null hypothesis

In this case, the null hypothesis is that the attribute is irrelevant and, hence, that the information gain for an infinitely large sample would be zero. We need to calculate the probability that, under the null hypothesis, a sample of size $v = n + p$ would exhibit the observed deviation from the expected distribution of positive and negative examples. We can measure the deviation by comparing the actual numbers of positive and negative examples in each subset, p_k and n_k , with the expected numbers, $\hat{p}_k$ and $\hat{n}_k$, assuming true irrelevance:

$$\hat{p}_k = p \times \frac{p_k + n_k}{p + n} \quad \hat{n}_k = n \times \frac{p_k + n_k}{p + n}.$$

A convenient measure of the total deviation is given by

$$\Delta = \sum_{k=1}^d \frac{(p_k - \hat{p}_k)^2}{\hat{p}_k} + \frac{(n_k - \hat{n}_k)^2}{\hat{n}_k}.$$

Under the null hypothesis, the value of Δ is distributed according to the χ^2 (chi-squared)

³ The gain will be strictly positive except for the unlikely case where all the proportions are *exactly* the same. (See Exercise 19.NNGA.)

distribution with $d - 1$ degrees of freedom. We can use a χ^2 statistics function to see if a particular Δ value confirms or rejects the null hypothesis. For example, consider the restaurant *Type* attribute, with four values and thus three degrees of freedom. A value of $\Delta = 7.82$ or more would reject the null hypothesis at the 5% level (and a value of $\Delta = 11.35$ or more would reject at the 1% level). Values below that lead to accepting the hypothesis that the attribute is irrelevant, and thus the associated branch of the tree should be pruned away. This is known as χ^2 **pruning**.

χ^2 pruning

With pruning, noise in the examples can be tolerated. Errors in the example's label (e.g., an example $(\mathbf{x}, \text{Yes})$ that should be $(\mathbf{x}, \text{No})$) give a linear increase in prediction error, whereas errors in the descriptions of examples (e.g., $\text{Price} = \$$ when it was actually $\text{Price} = \$\$$) have an asymptotic effect that gets worse as the tree shrinks down to smaller sets. Pruned trees perform significantly better than unpruned trees when the data contain a large amount of noise. Also, the pruned trees are often much smaller and hence easier to understand and more efficient to execute.

One final warning: You might think that χ^2 pruning and information gain look similar, so why not combine them using an approach called **early stopping**—have the decision tree algorithm stop generating nodes when there is no good attribute to split on, rather than going to all the trouble of generating nodes and then pruning them away. The problem with early stopping is that it stops us from recognizing situations where there is no one good attribute, but there are combinations of attributes that are informative. For example, consider the XOR function of two binary attributes. If there are roughly equal numbers of examples for all four combinations of input values, then neither attribute will be informative, yet the correct thing to do is to split on one of the attributes (it doesn't matter which one), and then at the second level we will get splits that are very informative. Early stopping would miss this, but generate-and-then-prune handles it correctly.

Early stopping

19.3.5 Broadening the applicability of decision trees

Decision trees can be made more widely useful by handling the following complications:

- **Missing data:** In many domains, not all the attribute values will be known for every example. The values might have gone unrecorded, or they might be too expensive to obtain. This gives rise to two problems: First, given a complete decision tree, how should one classify an example that is missing one of the test attributes? Second, how should one modify the information-gain formula when some examples have unknown values for the attribute? These questions are addressed in Exercise [19.MISS](#).
- **Continuous and multivalued input attributes:** For continuous attributes like *Height*, *Weight*, or *Time*, it may be that every example has a different attribute value. The information gain measure would give its highest score to such an attribute, giving us a shallow tree with this attribute at the root, and single-example subtrees for each possible value below it. But that doesn't help when we get a new example to classify with an attribute value that we haven't seen before.

Split point

A better way to deal with continuous values is a **split point** test—an inequality test on the value of an attribute. For example, at a given node in the tree, it might be the case that testing on $\text{Weight} > 160$ gives the most information. Efficient methods exist for finding good split points: start by sorting the values of the attribute, and then consider only split points that are between two examples in sorted order that have different

classifications, while keeping track of the running totals of positive and negative examples on each side of the split point. Splitting is the most expensive part of real-world decision tree learning applications.

For attributes that are not continuous and do not have a meaningful ordering, but have a large number of possible values (e.g., *Zipcode* or *CreditCardNumber*), a measure called the **information gain ratio** (see Exercise 19.GAIN) can be used to avoid splitting into lots of single-example subtrees. Another useful approach is to allow an **equality test** of the form $A = v_k$. For example, the test $Zipcode = 10002$ could be used to pick out a large group of people in this zip code in New York City, and to lump everyone else into the “other” subtree.

- **Continuous-valued output attribute:** If we are trying to predict a numerical output value, such as the price of an apartment, then we need a **regression tree** rather than a classification tree. A regression tree has at each leaf a linear function of some subset of numerical attributes, rather than a single output value. For example, the branch for two-bedroom apartments might end with a linear function of square footage and number of bathrooms. The learning algorithm must decide when to stop splitting and begin applying linear regression (see Section 19.6) over the attributes. The name **CART**, standing for Classification And Regression Trees, is used to cover both classes.

Regression tree

CART

A decision tree learning system for real-world applications must be able to handle all of these problems. Handling continuous-valued variables is especially important, because both physical and financial processes provide numerical data. Several commercial packages have been built that meet these criteria, and they have been used to develop thousands of fielded systems. In many areas of industry and commerce, decision trees are the first method tried when a classification method is to be extracted from a data set.

Decision trees have a lot going for them: ease of understanding, scalability to large data sets, and versatility in handling discrete and continuous inputs as well as classification and regression. However, they can have suboptimal accuracy (largely due to the greedy search), and if trees are very deep, then getting a prediction for a new example can be expensive in run time. Decision trees are also **unstable** in that adding just one new example can change the test at the root, which changes the entire tree. In Section 19.8.2 we will see that the **random forest model** can fix some of these issues.

Unstable

19.4 Model Selection and Optimization

Our goal in machine learning is to select a hypothesis that will optimally fit future examples. To make that precise we need to define “future example” and “optimal fit.”

First we will make the assumption that the future examples will be like the past. We call this the **stationarity** assumption; without it, all bets are off. We assume that each example E_j has the same prior probability distribution:

Stationarity

$$\mathbf{P}(E_j) = \mathbf{P}(E_{j+1}) = \mathbf{P}(E_{j+2}) = \cdots,$$

and is independent of the previous examples:

$$\mathbf{P}(E_j) = \mathbf{P}(E_j | E_{j-1}, E_{j-2}, \dots).$$

Examples that satisfy these equations are *independent and identically distributed* or **i.i.d.**

i.i.d.

CHAPTER 21

DEEP LEARNING

In which gradient descent learns multistep programs, with significant implications for the major subfields of artificial intelligence.

Deep learning

Layer

Neural network

Deep learning is a broad family of techniques for machine learning in which hypotheses take the form of complex algebraic circuits with tunable connection strengths. The word “deep” refers to the fact that the circuits are typically organized into many **layers**, which means that computation paths from inputs to outputs have many steps. Deep learning is currently the most widely used approach for applications such as visual object recognition, machine translation, speech recognition, speech synthesis, and image synthesis; it also plays a significant role in reinforcement learning applications (see Chapter 22).

Deep learning has its origins in early work that tried to model networks of neurons in the brain (McCulloch and Pitts, 1943) with computational circuits. For this reason, the networks trained by deep learning methods are often called **neural networks**, even though the resemblance to real neural cells and structures is superficial.

While the true reasons for the success of deep learning have yet to be fully elucidated, it has self-evident advantages over some of the methods covered in Chapter 19—particularly for high-dimensional data such as images. For example, although methods such as linear and logistic regression can handle a large number of input variables, the computation path from each input to the output is very short: multiplication by a single weight, then adding into the aggregate output. Moreover, the different input variables contribute independently to the output, without interacting with each other (Figure 21.1(a)). This significantly limits the expressive power of such models. They can represent only linear functions and boundaries in the input space, whereas most real-world concepts are far more complex.

Decision lists and decision trees, on the other hand, allow for long computation paths that can depend on many input variables—but only for a relatively small fraction of the possible input vectors (Figure 21.1(b)). If a decision tree has long computation paths for a significant fraction of the possible inputs, it must be exponentially large in the number of input variables. The basic idea of deep learning is to train circuits such that the computation paths are long, allowing all the input variables to interact in complex ways (Figure 21.1(c)). These circuit models turn out to be sufficiently expressive to capture the complexity of real-world data for many important kinds of learning problems.

Section 21.1 describes simple feedforward networks, their components, and the essentials of learning in such networks. Section 21.2 goes into more detail on how deep networks are put together, and Section 21.3 covers a class of networks called convolutional neural networks that are especially important in vision applications. Sections 21.4 and 21.5 go into more detail on algorithms for training networks from data and methods for improving

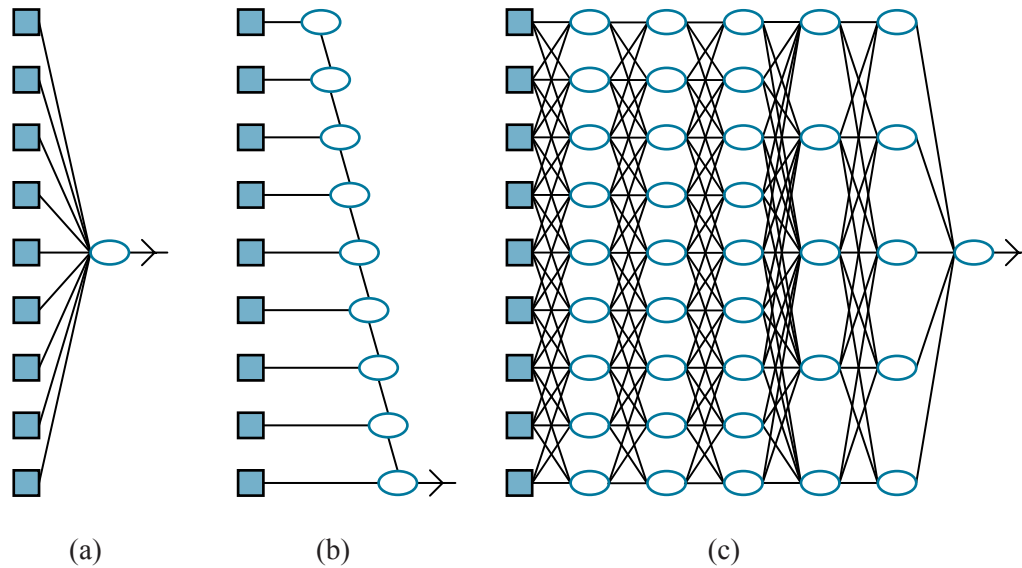


Figure 21.1 (a) A shallow model, such as linear regression, has short computation paths between inputs and output. (b) A decision list network (page 674) has some long paths for some possible input values, but most paths are short. (c) A deep learning network has longer computation paths, allowing each variable to interact with all the others.

generalization. Section 21.6 covers networks with recurrent structure, which are well suited for sequential data. Section 21.7 describes ways to use deep learning for tasks other than supervised learning. Finally, Section 21.8 surveys the range of applications of deep learning.

21.1 Simple Feedforward Networks

A **feedforward network**, as the name suggests, has connections only in one direction—that is, it forms a directed acyclic graph with designated input and output nodes. Each node computes a function of its inputs and passes the result to its successors in the network. Information flows through the network from the input nodes to the output nodes, and there are no loops. A **recurrent network**, on the other hand, feeds its intermediate or final outputs back into its own inputs. This means that the signal values within the network form a dynamical system that has internal state or memory. We will consider recurrent networks in Section 21.6.

Feedforward network

Recurrent network

Boolean circuits, which implement Boolean functions, are an example of feedforward networks. In a Boolean circuit, the inputs are limited to 0 and 1, and each node implements a simple Boolean function of its inputs, producing a 0 or a 1. In neural networks, input values are typically continuous, and nodes take continuous inputs and produce continuous outputs. Some of the inputs to nodes are **parameters** of the network; the network learns by adjusting the values of these parameters so that the network as a whole fits the training data.

21.1.1 Networks as complex functions

Each node within a network is called a **unit**. Traditionally, following the design proposed by McCulloch and Pitts, a unit calculates the weighted sum of the inputs from predecessor nodes

Unit

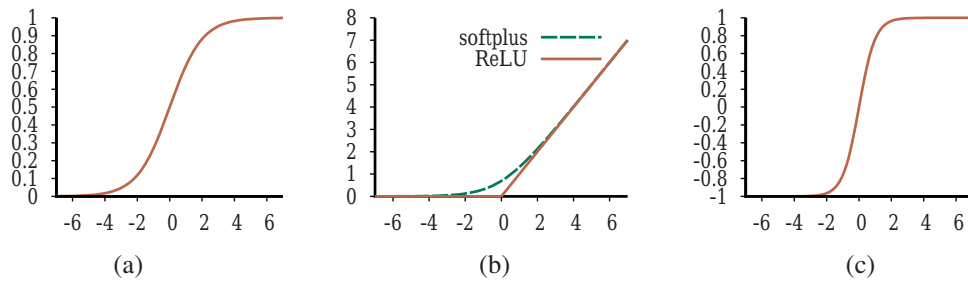


Figure 21.2 Activation functions commonly used in deep learning systems: (a) the logistic or sigmoid function; (b) the ReLU function and the softplus function; (c) the tanh function.

and then applies a nonlinear function to produce its output. Let a_j denote the output of unit j and let $w_{i,j}$ be the weight attached to the link from unit i to unit j ; then we have

$$a_j = g_j(\sum_i w_{i,j} a_i) \equiv g_j(in_j),$$

Activation function

where g_j is a nonlinear **activation function** associated with unit j and in_j is the weighted sum of the inputs to unit j .

As in Section 19.6.3 (page 679), we stipulate that each unit has an extra input from a dummy unit 0 that is fixed to +1 and a weight $w_{0,j}$ for that input. This allows the total weighted input in_j to unit j to be nonzero even when the outputs of the preceding layer are all zero. With this convention, we can write the preceding equation in vector form:

$$a_j = g_j(\mathbf{w}^\top \mathbf{x}) \quad (21.1)$$

where $\mathbf{w}$ is the vector of weights leading into unit j (including $w_{0,j}$) and $\mathbf{x}$ is the vector of inputs to unit j (including the +1).

The fact that the activation function is nonlinear is important because if it were not, any composition of units would still represent a linear function. The nonlinearity is what allows sufficiently large networks of units to represent arbitrary functions. The **universal approximation** theorem states that a network with just two layers of computational units, the first nonlinear and the second linear, can approximate any continuous function to an arbitrary degree of accuracy. The proof works by showing that an exponentially large network can represent exponentially many “bumps” of different heights at different locations in the input space, thereby approximating the desired function. In other words, sufficiently large networks can implement a lookup table for continuous functions, just as sufficiently large decision trees implement a lookup table for Boolean functions.

A variety of different activation functions are used. The most common are the following:

Sigmoid

- The logistic or **sigmoid** function, which is also used in logistic regression (see page 685):

$$\sigma(x) = 1/(1 + e^{-x}).$$

ReLU

- The **ReLU** function, whose name is an abbreviation for **rectified linear unit**:

$$\text{ReLU}(x) = \max(0, x).$$

Softplus

- The **softplus** function, a smooth version of the ReLU function:

$$\text{softplus}(x) = \log(1 + e^x).$$

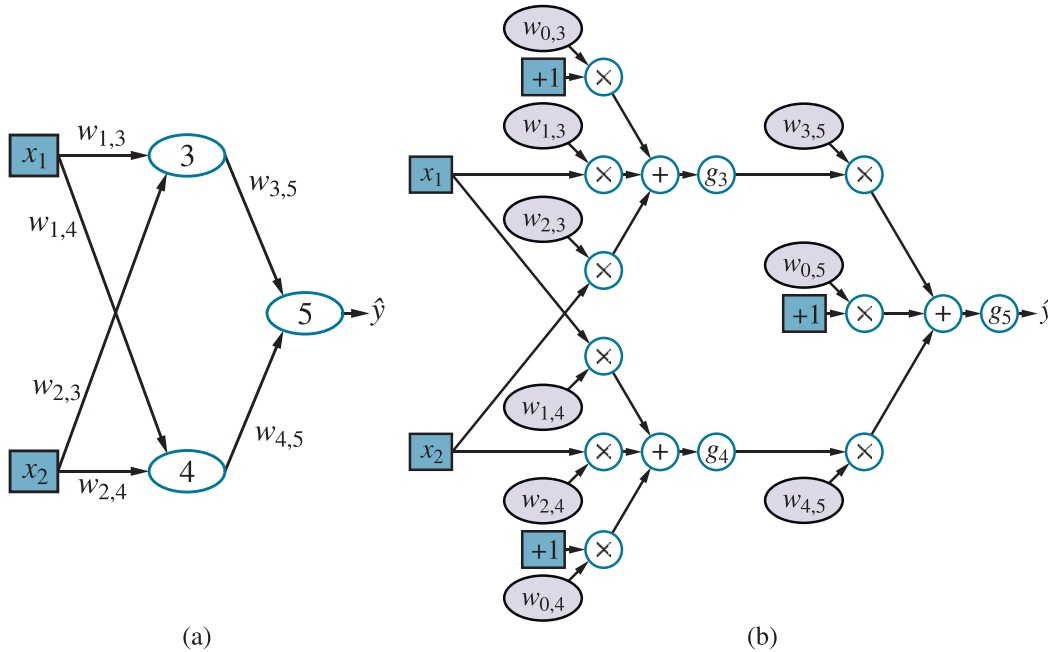


Figure 21.3 (a) A neural network with two inputs, one hidden layer of two units, and one output unit. Not shown are the dummy inputs and their associated weights. (b) The network in (a) unpacked into its full computation graph.

The derivative of the softplus function is the sigmoid function.

- The **tanh** function:

$$\tanh(x) = \frac{e^{2x} - 1}{e^{2x} + 1}.$$

Note that the range of $\tanh$ is $(-1, +1)$. $\tanh$ is a scaled and shifted version of the sigmoid, as $\tanh(x) = 2\sigma(2x) - 1$.

Tanh

These functions are shown in Figure 21.2. Notice that all of them are monotonically nondecreasing, which means that their derivatives g' are nonnegative. We will have more to say about the choice of activation function in later sections.

Coupling multiple units together into a network creates a complex function that is a composition of the algebraic expressions represented by the individual units. For example, the network shown in Figure 21.3(a) represents a function $h_{\mathbf{w}}(\mathbf{x})$, parameterized by weights $\mathbf{w}$, that maps a two-element input vector $\mathbf{x}$ to a scalar output value $\hat{y}$. The internal structure of the function mirrors the structure of the network. For example, we can write an expression for the output $\hat{y}$ as follows:

$$\begin{aligned} \hat{y} &= g_5(in_5) = g_5(w_{0,5} + w_{3,5}a_3 + w_{4,5}a_4) \\ &= g_5(w_{0,5} + w_{3,5}g_3(in_3) + w_{4,5}g_4(in_4)) \\ &= g_5(w_{0,5} + w_{3,5}g_3(w_{0,3} + w_{1,3}x_1 + w_{2,3}x_2) \\ &\quad + w_{4,5}g_4(w_{0,4} + w_{1,4}x_1 + w_{2,4}x_2)). \end{aligned} \quad (21.2)$$

Thus, we have the output $\hat{y}$ expressed as a function $h_{\mathbf{w}}(\mathbf{x})$ of the inputs and the weights.

Computation graph
Dataflow graph

Figure 21.3(a) shows the traditional way a network might be depicted in a book on neural networks. A more general way to think about the network is as a **computation graph** or **dataflow graph**—essentially a circuit in which each node represents an elementary computation. Figure 21.3(b) shows the computation graph corresponding to the network in Figure 21.3(a); the graph makes each element of the overall computation explicit. It also distinguishes between the inputs (in blue) and the weights (in light mauve): the weights can be adjusted to make the output $\hat{y}$ agree more closely with the true value y in the training data. Each weight is like a volume control knob that determines how much the next node in the graph hears from that particular predecessor in the graph.

Just as Equation (21.1) described the operation of a unit in vector form, we can do something similar for the network as a whole. We will generally use $\mathbf{W}$ to denote a weight matrix; for this network, $\mathbf{W}^{(1)}$ denotes the weights in the first layer ($w_{1,3}$, $w_{1,4}$, etc.) and $\mathbf{W}^{(2)}$ denotes the weights in the second layer ($w_{3,5}$ etc.). Finally, let $\mathbf{g}^{(1)}$ and $\mathbf{g}^{(2)}$ denote the activation functions in the first and second layers. Then the entire network can be written as follows:

$$h_{\mathbf{w}}(\mathbf{x}) = \mathbf{g}^{(2)}(\mathbf{W}^{(2)}\mathbf{g}^{(1)}(\mathbf{W}^{(1)}\mathbf{x})). \quad (21.3)$$

Like Equation (21.2), this expression corresponds to a computation graph, albeit a much simpler one than the graph in Figure 21.3(b): here, the graph is simply a chain with weight matrices feeding into each layer.

Fully connected

The computation graph in Figure 21.3(b) is relatively small and shallow, but the same idea applies to all forms of deep learning: we construct computation graphs and adjust their weights to fit the data. The graph in Figure 21.3(b) is also **fully connected**, meaning that every node in each layer is connected to every node in the next layer. This is in some sense the default, but we will see in Section 21.3 that choosing the connectivity of the network is also important in achieving effective learning.

21.1.2 Gradients and learning

Output layer

Hidden layer

In Section 19.6, we introduced an approach to supervised learning based on **gradient descent**: calculate the gradient of the loss function with respect to the weights, and adjust the weights along the gradient direction to reduce the loss. (If you have not already read Section 19.6, we recommend strongly that you do so before continuing.) We can apply exactly the same approach to learning the weights in computation graphs. For the weights leading into units in the **output layer**—the ones that produce the output of the network, the gradient calculation is essentially identical to the process in Section 19.6. For weights leading into units in the **hidden layers**, which are not directly connected to the outputs, the process is only slightly more complicated.

For now, we will use the squared loss function, L_2 , and we will calculate the gradient for the network in Figure 21.3 with respect to a single training example $(\mathbf{x}, y)$. (For multiple examples, the gradient is just the sum of the gradients for the individual examples.) The network outputs a prediction $\hat{y} = h_{\mathbf{w}}(\mathbf{x})$ and the true value is y , so we have

$$\text{Loss}(h_{\mathbf{w}}) = L_2(y, h_{\mathbf{w}}(\mathbf{x})) = \|y - h_{\mathbf{w}}(\mathbf{x})\|^2 = (y - \hat{y})^2.$$

To compute the gradient of the loss with respect to the weights, we need the same tools of calculus we used in Chapter 19—principally the **chain rule**, $\partial g(f(x))/\partial x = g'(f(x)) \partial f(x)/\partial x$. We'll start with the easy case: a weight such as $w_{3,5}$ that is connected to the output unit. We operate directly on the network-defining expressions from Equation (21.2):

$$\begin{aligned}
\frac{\partial}{\partial w_{3,5}} \text{Loss}(h_{\mathbf{w}}) &= \frac{\partial}{\partial w_{3,5}} (y - \hat{y})^2 = -2(y - \hat{y}) \frac{\partial \hat{y}}{\partial w_{3,5}} \\
&= -2(y - \hat{y}) \frac{\partial}{\partial w_{3,5}} g_5(\text{in}_5) = -2(y - \hat{y}) g'_5(\text{in}_5) \frac{\partial}{\partial w_{3,5}} \text{in}_5 \\
&= -2(y - \hat{y}) g'_5(\text{in}_5) \frac{\partial}{\partial w_{3,5}} (w_{0,5} + w_{3,5} a_3 + w_{4,5} a_4) \\
&= -2(y - \hat{y}) g'_5(\text{in}_5) a_3.
\end{aligned} \tag{21.4}$$

The simplification in the last line follows because $w_{0,5}$ and $w_{4,5} a_4$ do not depend on $w_{3,5}$, nor does the coefficient of $w_{3,5}$, a_3 .

The slightly more difficult case involves a weight such as $w_{1,3}$ that is not directly connected to the output unit. Here, we have to apply the chain rule one more time. The first few steps are identical, so we omit them:

$$\begin{aligned}
\frac{\partial}{\partial w_{1,3}} \text{Loss}(h_{\mathbf{w}}) &= -2(y - \hat{y}) g'_5(\text{in}_5) \frac{\partial}{\partial w_{1,3}} (w_{0,5} + w_{3,5} a_3 + w_{4,5} a_4) \\
&= -2(y - \hat{y}) g'_5(\text{in}_5) w_{3,5} \frac{\partial}{\partial w_{1,3}} a_3 \\
&= -2(y - \hat{y}) g'_5(\text{in}_5) w_{3,5} \frac{\partial}{\partial w_{1,3}} g_3(\text{in}_3) \\
&= -2(y - \hat{y}) g'_5(\text{in}_5) w_{3,5} g'_3(\text{in}_3) \frac{\partial}{\partial w_{1,3}} \text{in}_3 \\
&= -2(y - \hat{y}) g'_5(\text{in}_5) w_{3,5} g'_3(\text{in}_3) \frac{\partial}{\partial w_{1,3}} (w_{0,3} + w_{1,3} x_1 + w_{2,3} x_2) \\
&= -2(y - \hat{y}) g'_5(\text{in}_5) w_{3,5} g'_3(\text{in}_3) x_1.
\end{aligned} \tag{21.5}$$

So, we have fairly simple expressions for the gradient of the loss with respect to the weights $w_{3,5}$ and $w_{1,3}$.

If we define $\Delta_5 = 2(\hat{y} - y) g'_5(\text{in}_5)$ as a sort of “perceived error” at the point where unit 5 receives its input, then the gradient with respect to $w_{3,5}$ is just $\Delta_5 a_3$. This makes perfect sense: if Δ_5 is positive, that means $\hat{y}$ is too big (recall that g' is always nonnegative); if a_3 is also positive, then increasing $w_{3,5}$ will only make things worse, whereas if a_3 is negative, then increasing $w_{3,5}$ will reduce the error. The magnitude of a_3 also matters: if a_3 is small for this training example, then $w_{3,5}$ didn’t play a major role in producing the error and doesn’t need to be changed much.

If we also define $\Delta_3 = \Delta_5 w_{3,5} g'_3(\text{in}_3)$, then the gradient for $w_{1,3}$ becomes just $\Delta_3 x_1$. Thus, the perceived error at the input to unit 3 is the perceived error at the input to unit 5, multiplied by information along the path from 5 back to 3. This phenomenon is completely general, and gives rise to the term **back-propagation** for the way that the error at the output is passed back through the network.

Back-propagation

Another important characteristic of these gradient expressions is that they have as factors the local derivatives $g'_j(\text{in}_j)$. As noted earlier, these derivatives are always nonnegative, but they can be very close to zero (in the case of the sigmoid, softplus, and tanh functions) or exactly zero (in the case of ReLUs), if the inputs from the training example in question

Vanishing gradient

happen to put unit j in the flat operating region. If the derivative g'_j is small or zero, that means that changing the weights leading into unit j will have a negligible effect on its output. As a result, deep networks with many layers may suffer from a **vanishing gradient**—the error signals are extinguished altogether as they are propagated back through the network. Section 21.3.3 provides one solution to this problem.

We have shown that gradients in our tiny example network are simple expressions that can be computed by passing information back through the network from the output units. It turns out that this property holds more generally. In fact, as we show in Section 21.4.1, the gradient computations for *any* feedforward computation graph have the same structure as the underlying computation graph. This property follows straightforwardly from the rules of differentiation.

Automatic differentiation

Reverse mode

We have shown the gory details of a gradient calculation, but worry not: there is no need to redo the derivations in Equations (21.4) and (21.5) for each new network structure! All such gradients can be computed by the method of **automatic differentiation**, which applies the rules of calculus in a systematic way to calculate gradients for any numeric program.¹ In fact, the method of back-propagation in deep learning is simply an application of **reverse mode** differentiation, which applies the chain rule “from the outside in” and gains the efficiency advantages of dynamic programming when the network in question has many inputs and relatively few outputs.

End-to-end learning

All of the major packages for deep learning provide automatic differentiation, so that users can experiment freely with different network structures, activation functions, loss functions, and forms of composition without having to do lots of calculus to derive a new learning algorithm for each experiment. This has encouraged an approach called **end-to-end learning**, in which a complex computational system for a task such as machine translation can be composed from several trainable subsystems; the entire system is then trained in an end-to-end fashion from input/output pairs. With this approach, the designer need have only a vague idea about how the overall system should be structured; there is no need to know in advance exactly what each subsystem should do or how to label its inputs and outputs.

21.2 Computation Graphs for Deep Learning

We have established the basic ideas of deep learning: represent hypotheses as computation graphs with tunable weights and compute the gradient of the loss function with respect to those weights in order to fit the training data. Now we look at how to put together computation graphs. We begin with the input layer, which is where the training or test example $\mathbf{x}$ is encoded as values of the input nodes. Then we consider the output layer, where the outputs $\hat{\mathbf{y}}$ are compared with the true values $\mathbf{y}$ to derive a learning signal for tuning the weights. Finally, we look at the hidden layers of the network.

21.2.1 Input encoding

The input and output nodes of a computational graph are the ones that connect directly to the input data $\mathbf{x}$ and the output data $\mathbf{y}$. The encoding of input data is usually straightforward, at least for the case of factored data where each training example contains values for n input

¹ Automatic differentiation methods were originally developed in the 1960s and 1970s for optimizing the parameters of systems defined by large, complex Fortran programs.